INFORME FINAL DESAFÍO II

Samuel Mauricio Bastidas Jaramillo (1138525766)

Sergio Alejandro Cabrera Narvaez (1086922688)

1. Contexto del problema:

Ude@Book es una red social por consola cuya base son los usuarios. Cada usuario tiene datos de perfil (nombre de usuario, clave, correo, fecha de nacimiento, ciudad, país, fecha de inscripción y ruta de su foto de perfil), una colección de amigos, álbumes de fotos, mensajes enviados y recibidos, y solicitudes de amistad. Las reglas para el diseño son las siguientes:

- La amistad es simétrica: solo existe después de que el receptor acepta una solicitud de otra persona.
- Un álbum tiene tres niveles de privacidad (público, amigos y privado). Al visitar el perfil de un amigo solo se muestran los públicos y los de amigos.
- Los mensajes tienen fecha de envío, un estado leído o no leído, un máximo de 512 caracteres y solo pueden enviarse a amigos. No se aceptan mensajes vacíos ni con saltos de línea.
- No se envían solicitudes a uno mismo, a quien ya es amigo, ni a quien ya tiene una solicitud pendiente nuestra o nos la envió a nosotros.
- Al ingresar se muestran las fotos de perfil de tres "mejores amigos" elegidos al azar con probabilidad ponderada por la cantidad de amigos en común (peso 1 con 10 o menos, peso 2.5 de 11 a 20, y peso 3.5 con más de 20). Esas tres fotos pueden cambiarse por otro grupo, una sola vez por sesión.
- El sistema sugiere hasta K amigos (K entre 1 y 20) tomando entre los amigos de amigos a quienes tienen más amigos en común.
- Cada funcionalidad debe reportar su consumo de recursos: memoria (dinámica más variables locales) e iteraciones.

1.2 Restricciones y su efecto en la solución:

El enunciado prohíbe contenedores de la STL, structs, funciones de ANSI C y librerías de terceros que resuelvan cosas que puedo hacer con mi equipo, y exige memoria dinámica, programación orientada a objetos con al menos tres clases y diseño multi-archivo (.h y .cpp). También prohíbe usar el almacenamiento permanente como área de procesamiento. Esto produjo cuatro decisiones de fondo:

a. Todas las colecciones son de creación propia (la plantilla Lista<T> con su Nodo<T>) y el único componente externo usado sson las bibliotecas estándar de C++ para números aleatorios, fechas del sistema y manejo de archivos de texto.
b. Las cadenas son arreglos de caracteres dinámicos (char*) manejados por una clase utilitaria propia, Cadenas, sin la clase string.
c. Los datos se cargan completos a memoria al inicio. Los archivos solo se usan para cargar y para guardar; nunca para buscar ni procesar.
d. La eficiencia es un criterio central: se eligió la estructura de cada dato según la operación que más se repite sobre él.

1.3 Clases de la solución y responsabilidades

La solución tiene doce clases

RedSocial es la clase controladora. Es dueña de la lista de usuarios, de un índice directo por identificador (usuariosPorId, un arreglo dinámico de punteros a Usuario) y del grafo de amistades. Coordina la carga y el guardado de los archivos, la autenticación, el envío y la respuesta de solicitudes, el envío de mensajes, las sugerencias, la selección de mejores amigos y los likes. Define códigos de resultado (SOLICITUD_OK, MENSAJE_VACIO, etc.) para que la lógica de negocio viva en ella y el menú solo informe al usuario. Su constructor de copia y su operador de asignación son privados porque una red social es única.

- Usuario guarda los datos personales como cadenas dinámicas y cinco listas: amigos (no es dueño de ellos), álbumes (es dueño), mensajes enviados (es dueño), mensajes recibidos (solo guarda punteros) y solicitudes recibidas (es dueño). Tiene un constructor de copia profunda de los datos personales, operador de asignación y operador de igualdad por identificador.
- *Album agrupa las fotos de un usuario con nombre, ruta de almacenamiento y privacidad, y es dueño de sus fotos (Lista<Foto>*). Foto guarda la ruta, el tamaño en bytes, la fecha de captura y el contador de likes, además de una bandera de sesión (likeActivo) que permite alternar el like. Foto y Mensaje sobrecargan el operador de salida (<<) para imprimirse.
- Mensaje guarda identificadores de emisor y receptor (no copias de los nombres), contenido, fecha y estado de lectura. Sobrecarga el operador < para ordenar por fecha. SolicitudAmistad guarda emisor, receptor y un estado numérico (pendiente, aceptada o rechazada).
- GrafoAmistad modela la red de amistades con una matriz de adyacencia dinámica de booleanos (bool**) de N por N. Ofrece consulta de amistad en tiempo constante, agregar amistad simétrica, amigos en común entre dos usuarios y un conteo de amigos en común de un usuario contra todos los demás en una sola pasada.
- Lista<T> es una lista enlazada simple con puntero al último nodo y Nodo<T> es su nodo; Lista es clase amiga de Nodo para manipular los enlaces sin exponerlos. Lista sobrecarga el operador de indexación ([]).
- MonitorRecursos acumula los bytes dinámicos asignados y liberados, las iteraciones y los usos de componentes externos. Existe una única instancia global (monitorGlobal). Cadenas y Fecha son clases utilitarias estáticas (con constructor privado) para manejo de cadenas y de fechas.

1.4 Archivos de datos y persistencia

Se usan cinco archivos de texto con separador "|" y una línea de encabezado: usuarios.txt, amistades.txt, albumesfotos.txt, solicitudes.txt y mensajes.txt. Los campos son los de las plantillas entregadas: por ejemplo, un mensaje se guarda como ID_Mensaje|ID_Emisor|ID_Receptor|Fecha_Envio|Leido|Contenido, con el contenido al final para que pueda contener el carácter "|". Un álbum sin fotos se guarda en una línea con los campos de foto vacíos.

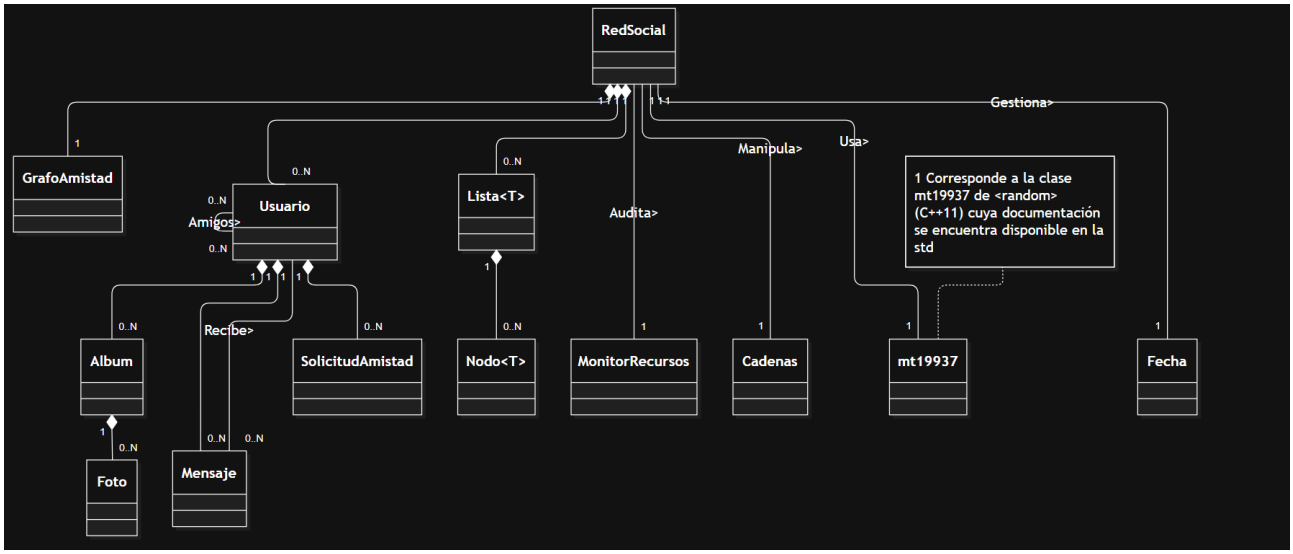
La estrategia de guardado distingue dos tipos de cambio:

Altas (un mensaje nuevo, una solicitud nueva, una amistad nueva): se añade una sola línea al final del archivo correspondiente, con costo independiente del tamaño del archivo.

Cambios sobre registros existentes (marcar mensajes como leídos, responder una solicitud, dar o quitar un like): se reescribe el archivo afectado.

Si una escritura falla, el archivo queda marcado como pendiente (banderas amistadesPendientes, albumesPendientes, solicitudesPendientes y mensajesPendientes) y se vuelve a intentar al cerrar sesión o al salir, mediante guardarDatos.

2. DIAGRAMA DE CLASES



3. ALGORITMOS ESENCIALES

Se describe la lógica de alto nivel de las tareas

3.1 Carga de datos

La carga se hace en un orden fijo porque cada archivo depende del anterior. Primero se lee usuarios.txt en dos pasadas: la primera solo cuenta los usuarios para saber la capacidad N; con N se reservan, con tamaño exacto, el índice por identificador y la matriz de adyacencia; la segunda pasada valida y crea cada usuario. Después se leen las amistades (por cada par se consulta la matriz en tiempo constante y se actualizan la matriz y las dos listas de amigos), luego álbumes y fotos, solicitudes y mensajes. Cada línea se lee en un búfer fijo de 2048 caracteres, sin reservar memoria por línea, y se valida: número de campos, identificadores dentro de 0 a N-1, fechas válidas, y usuario repetido. Ante un error se informa el archivo y la línea y la carga se detiene.

3.2 Selección de los tres mejores amigos (ruleta ponderada sin repetición)

1. Con una sola pasada del grafo se obtiene la cantidad de amigos en común del usuario con todos los demás (conteo de amigos en común contra todos).
2. Se recorren los amigos del usuario; si se está alternando se descartan los tres mostrados antes. A cada candidato se le asigna un peso entero: 2 (10 o menos en común), 5 (de 11 a 20) y 7 (más de 20). Son los pesos 1, 2.5 y 3.5 multiplicados por 2, de modo que toda la aritmética sea entera y exacta.
3. Se acumula la suma de pesos. Para cada uno de los tres sorteos se genera un número aleatorio entre 0 y la suma menos uno y se recorren los candidatos acumulando pesos hasta que el número cae dentro de un tramo; ese candidato gana.
4. Para no repetir, el ganador se retira en tiempo constante: se resta su peso de la suma total y se coloca en su lugar al último candidato. Así los tres elegidos son distintos sin descartar sorteos.
5. Se guarda el grupo mostrado para excluirlo si el usuario pide alternar, lo cual solo se permite una vez por sesión. Si hay menos de tres candidatos, se muestran los que existan.

3.3 Sugerencias de amistad (Top-K)

1. Se valida que K esté entre 1 y 20.

2. Se calcula, en una sola pasada, el número de amigos en común del usuario con cada otro usuario: por cada amigo del usuario se suma uno al contador de cada amigo de ese amigo. Un usuario con al menos un amigo en común es, por definición, amigo de un amigo.
3. Se recorren todos los usuarios y se descartan el propio usuario, los que ya son sus amigos (consulta en tiempo constante) y los de conteo cero.
4. Se mantiene un arreglo ordenado de tamaño K con los mejores candidatos. Un candidato nuevo entra al final si el arreglo no está lleno o reemplaza al último si lo supera, y luego se desplaza hacia su posición por inserción ordenada. Nunca se ordena la lista completa de candidatos.
5. Los K elegidos se entregan en una lista, de mayor a menor coincidencia. El menú permite elegir uno y enviarle la solicitud, que pasa por todas las validaciones de enviarSolicitud.

3.4 Mensajes

Cada lista de mensajes (enviados y recibidos) se mantiene siempre ordenada por fecha ascendente. Al insertar un mensaje se recorre la lista hasta la primera fecha estrictamente mayor y se inserta en esa posición; con fechas iguales se conserva el orden de llegada. Esa invariante permite dos cosas:

- Buscar por rango: se recorre la lista de recibidos desde el inicio, se descartan los anteriores a la fecha inicial, se agregan los que están dentro del rango (ambos extremos incluidos) y el recorrido se detiene en el primer mensaje posterior a la fecha final. La consulta no cambia el estado de lectura.
- Leer mensajes nuevos: se recorre la lista de recibidos, se recogen los no leídos (ya salen ordenados por fecha) y se marcan como leídos; como cambió un registro existente, se reescribe mensajes.txt.

Para enviar un mensaje, RedSocial valida en este orden: no vacío, a lo sumo 512 caracteres, sin saltos de línea, destinatario existente y distinto del emisor, y que sean amigos. Luego se crea el mensaje (el emisor es su dueño), se inserta ordenado en la lista de enviados del emisor y en la de recibidos del receptor (que solo guarda el puntero) y se añade una línea al archivo.

3.5 Solicitudes de amistad

Para enviar una solicitud se valida que el receptor exista, que no sea el propio emisor, que no sean amigos y que no haya una solicitud pendiente en ninguno de los dos sentidos (las rechazadas no bloquean un nuevo envío). Al aceptar, se cambia el estado de la solicitud, se agrega la arista simétrica a la matriz, se actualizan las dos listas de amigos y se añade una línea a amistades.txt; el estado nuevo de la solicitud se reescribe en solicitudes.txt.

3.6 Visita al perfil y carrusel de fotos

Al visitar el perfil de un amigo se verifica que exista y que sea amigo, y se listan solo sus álbumes públicos o de amigos. Al abrir un álbum, un índice parte en cero y se muestra la foto activa con su ruta y sus likes. "Siguiente" avanza con aritmética modular, (índice + 1) mod total, y "Previa" retrocede con (índice - 1 + total) mod total, de modo que el recorrido es circular. "Dar o quitar like" alterna el like de esa foto para el usuario de la sesión: si no lo tenía suma uno, si ya lo tenía resta uno. RedSocial recuerda las fotos tocadas en una lista corta y, al cerrar sesión, reinicia solo esas fotos en vez de recorrer todos los álbumes de la red.

3.7 Medición del consumo de recursos

MonitorRecursos expone operaciones para registrar asignaciones y liberaciones (con sizeof del tipo reservado), iteraciones y usos de componentes externos (por ejemplo, abrir un archivo, generar un número aleatorio). Cada new y cada delete de las clases y de las listas notifican al monitor, de modo que la memoria dinámica activa es siempre la diferencia entre lo asignado y lo liberado. Los bucles de recorrido, búsqueda y ordenamiento registran sus iteraciones; cuando el costo es conocido (por ejemplo, una fila completa de la matriz) se registra de una vez, y por fila

para no desbordar el contador entero. Al terminar cada funcionalidad del menú se imprimen las iteraciones, los usos de componentes externos, la memoria dinámica, la memoria de variables locales y parámetros (calculada con sizeof en cada función del menú más el tamaño del objeto RedSocial como base fija), la memoria total y los bloques dinámicos activos, y luego se reinician los contadores de iteraciones para la siguiente funcionalidad.

4. EFICIENCIA: ESTRUCTURAS ELEGIDAS Y OPTIMIZACIONES

4.1 Estructuras de datos y costo de las operaciones principales

Las estructuras se eligieron según la operación que más se repite sobre cada dato. En lo que sigue, N es el número de usuarios, F el número de amigos de un usuario y M el número de mensajes de una lista.

- Consultar si dos usuarios son amigos cuesta una sola operación, O(1), por la matriz de adyacencia. Con una lista de amigos costaría O(F).
- Buscar un usuario por identificador cuesta O(1) con el índice usuariosPorId. Una búsqueda secuencial en la lista de usuarios costaría O(N).
- Insertar al final de cualquier lista cuesta O(1), porque Lista conserva un puntero al último nodo. Sin ese puntero cada inserción costaría O(tamaño).
- Contar los amigos en común de un usuario con todos los demás cuesta N + F*N, en lugar de N*N si se calculara por separado contra cada candidato.
- Las listas son enlazadas en vez de arreglos porque amigos, mensajes, solicitudes y fotos crecen durante la ejecución y no se conocen de antemano; así se asigna y libera solo lo necesario. La matriz de adyacencia, en cambio, es un arreglo de N por N porque su tamaño sí se conoce desde la primera pasada de la carga y se accede por posición.
- Los parámetros estructurados se pasan siempre por referencia (por ejemplo, las listas de resultado se reciben por referencia y los getters devuelven referencias constantes a las listas), para no copiar nodos y no distorsionar el conteo de memoria e iteraciones.

4.2 Consumo de memoria

Con N = 1000, la matriz de adyacencia ocupa N por N bytes (1,000,000 bytes, cerca de 1 MB) más N punteros de fila (8,000 bytes); el índice por identificador ocupa otros 8,000 bytes. Cada elemento de una lista cuesta un nodo de 16 bytes (dato más puntero). El costo de la matriz es deliberado: se cambia espacio por las consultas de amistad en O(1) y por el conteo de amigos en común, que son las dos operaciones centrales de la red. Toda esa memoria se contabiliza en MonitorRecursos y se libera por completo al destruir la red social.

5. PROBLEMAS DE DESARROLLO

5.1 Propiedad de los mensajes y liberación doble

Un mensaje aparece en dos listas: la de enviados de su emisor y la de recibidos de su receptor. En una primera versión ambos usuarios liberaban el mensaje al destruirse, lo que provocaba una liberación doble de memoria. Se resolvió definiendo propiedad explícita: el emisor es dueño del mensaje y el receptor solo guarda un puntero. Esto se refleja en el diagrama con dos relaciones distintas entre Usuario y Mensaje: una de composición (el emisor lo contiene) y una asociación nombrada ("Recibe") para el receptor. La misma regla se aplicó a amigos (Usuario no es dueño de sus amigos), a álbumes (el usuario es dueño) y a fotos (el álbum es dueño), con un orden de destrucción documentado.

5.2 Copia profunda de cadenas

Como los datos personales son char*, una copia superficial haría que dos objetos compartan la misma memoria y la liberen dos veces. Se implementaron constructores de copia y operadores de asignación con copia profunda en Usuario, Album, Foto, Mensaje y SolicitudAmistad, y la asignación duplica primero el nuevo contenido y libera después el anterior, de modo que una falla no deja el objeto en un estado inválido. En Usuario la copia solo duplica los datos personales y deja vacías las listas, porque copiar las listas de punteros duplicaría la propiedad; esto se documentó en la clase.

5.3 Diseño del diagrama de clases

Fue la parte que más iteraciones requirió. Los problemas concretos fueron: distinguir correctamente composición ("es parte de") de la asociación simple, que es la única que lleva nombre; evitar relaciones redundantes o transitivas (por ejemplo, relacionar RedSocial directamente con Foto cuando ya existe la cadena RedSocial, Usuario, Album, Foto); decidir cómo representar las clases utilitarias Cadenas, Fecha y MonitorRecursos, que usan casi todas las demás y por eso no pertenecen a una sola clase; representar los componentes externos de C++ (generador aleatorio, reloj, flujos de archivo) como cajas sin atributos ni métodos con su biblioteca y versión; y mantener el diagrama sincronizado con el código a medida que cambiaban las clases.

5.4 Eliminación de atributos y métodos redundantes

Una versión inicial de Mensaje y SolicitudAmistad guardaba además los nombres del emisor y del receptor como cadenas. Eso duplicaba datos que ya estaban en Usuario, ocupaba memoria innecesaria y obligaba a mantenerlos sincronizados. Se reemplazaron por los identificadores. También se eliminaron constructores y métodos que ningún flujo del programa usaba, para que el diagrama y el código reflejen solo lo que realmente se necesita.

5.5 Desbordamiento del contador de iteraciones

Al contar la inicialización de la matriz N por N en una sola llamada al monitor, el valor N*N podía exceder el rango de un entero. Se resolvió registrando las iteraciones fila por fila (N + 1 por fila) y usando un acumulador de tipo size_t.

5.6 Formato de archivos y mensajes con el separador

Un mensaje que contuviera el carácter "|" corrompía la lectura por campos, y uno con saltos de línea rompía la regla de un registro por línea. Se resolvió colocando el contenido como último campo (el último campo toma todo el resto de la línea) y rechazando los mensajes con saltos de línea con un código de resultado propio. Además, la lectura valida longitud de línea, número de campos y rangos, e informa el archivo y la línea del error.

5.7 Constantes estáticas y referencias

Las constantes de resultado de RedSocial (por ejemplo, SOLICITUD_OK) son miembros estáticos constantes. Pasarlas a funciones que reciben referencias constantes puede generar errores de enlace; por eso siempre se usan por valor, y se dejó documentado en la clase.

5.8 Costo de persistir cada cambio

Reescribir un archivo completo para cada mensaje nuevo habría sido lineal en el tamaño del archivo. Por eso las altas se añaden al final (costo constante) y solo los cambios sobre registros existentes reescriben el archivo; las banderas de pendiente evitan perder datos cuando una escritura falla.

6. EVOLUCIÓN DE LA SOLUCIÓN Y CAMBIOS RESPECTO AL INFORME PRELIMINAR

El informe preliminar fijó la base del diseño (Lista<T> y Nodo<T> con Lista como amiga de Nodo, matriz de adyacencia en GrafoAmistad, MonitorRecursos transversal y las clases de dominio) y se mantuvo. Durante la implementación se hicieron estos cambios, ordenados de mayor a menor impacto:

a. Índice por identificador. Se añadió usuariosPorId a RedSocial para eliminar la búsqueda lineal por identificador.
b. Conteo de amigos en común en una pasada. Se agregó la operación de conteo contra todos los usuarios en GrafoAmistad, que usan las sugerencias y los mejores amigos
c. Sugerencias con arreglo Top-K. En lugar de ordenar la lista completa de candidatos, como planteaba el preliminar, se mantiene un arreglo ordenado de K posiciones con inserción ordenada.
d. Lista con puntero al último nodo. Se añadió para que insertar al final sea O(1).
e. Mensajes ordenados por fecha. Las listas de mensajes se mantienen ordenadas, lo que permite cortar la búsqueda por rango en la primera fecha posterior a la final.
f. Identificadores en lugar de nombres. Mensaje y SolicitudAmistad guardan identificadores de emisor y receptor, no cadenas con los nombres.
g. Persistencia. La estrategia final distingue entre altas (se añade una línea) y modificaciones (se reescribe el archivo), con banderas de pendiente y sincronización final. No se usan archivos temporales que se renombren, como se había propuesto, porque las escrituras ya son pequeñas o se reintentan con las banderas.
h. Alternancia de likes por sesión. Se añadió el estado de sesión en Foto y la lista de fotos tocadas, para reiniciar al cerrar sesión solo lo que cambió.
i. Pesos enteros. Los pesos 1, 2.5 y 3.5 se multiplican por 2 (2, 5 y 7) para usar solo enteros.